pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

pandas: Combining and Loading, Part 1

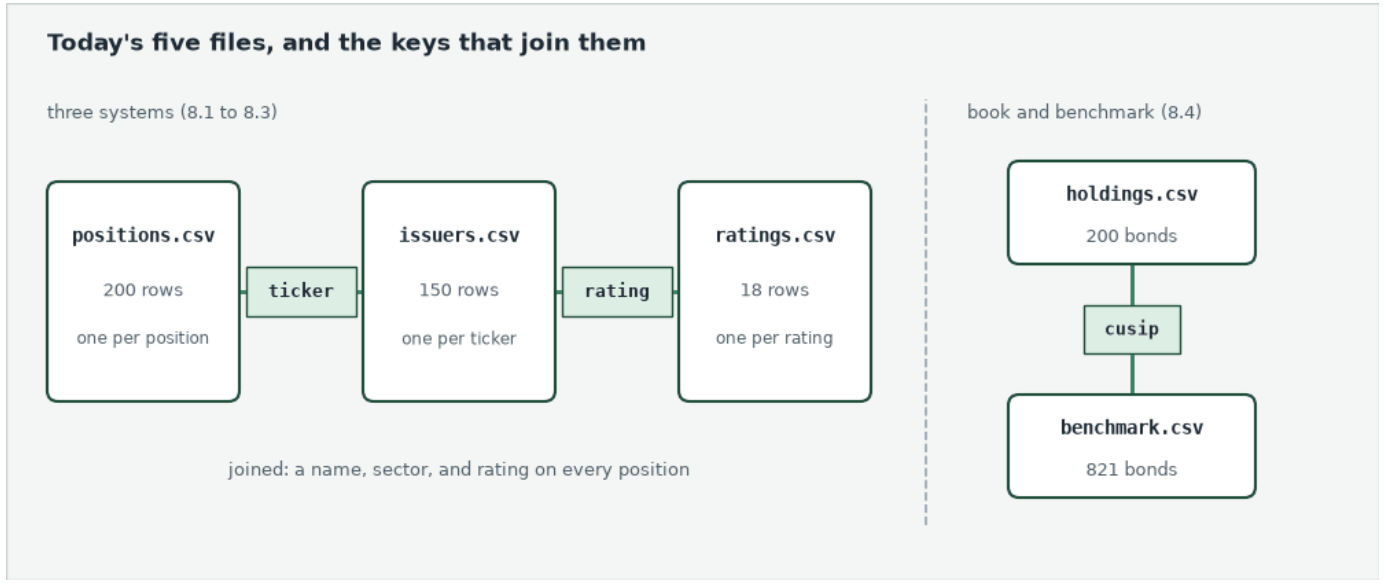
WEEK 2, DAY 3, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's twelfth note

Today's data comes from three systems: positions, issuer details, and the rating scale. Put a name, a sector, and a rating on every position, and show me none was lost or doubled. Which benchmark bonds do we hold, and what weight do they carry? Then the book by sector, in money, beside the benchmark.



Three pieces, joined by two keys: load them first.

The issuers are invented.

8.1 Positions, issuers, and the rating scale

```
positions = pd.read_csv(data_folder + 'positions.csv')
print(positions.shape)
positions.head(3)
(200, 6)
       cusip ticker  coupon    maturity    price  par_held
0  99080CAE8   BEZE   7.000  2032-06-18  106.727   5000000
1  99049XAE2   BEZH   8.125  2029-04-17   97.556    250000
2  99039NAC0   BEZL   5.125  2030-06-03   98.730   1500000
```

```
issuers = pd.read_csv(data_folder + 'issuers.csv')
print(issuers.shape, issuers['ticker'].nunique())
issuers.head(3)
(150, 5) 150
  ticker           issuer          sector country rating
0   QVNE  Quorvane Energy          Energy      US    BBB
1   DNMR   Dunmarrow Rail  Transportation      US    BB+
2   PLWK   Pellwick Paper       Materials      US     A-
```

```
print('Tickers held:', positions['ticker'].nunique())
Tickers held: 115
```

```
ratings = pd.read_csv(data_folder + 'ratings.csv')
print(ratings.shape)
ratings.head(11)
(18, 4)
   rating  score bucket             grade
0     AAA      1    AAA  Investment Grade
1     AA+      2     AA  Investment Grade
2      AA      3     AA  Investment Grade
3     AA-      4     AA  Investment Grade
4      A+      5      A  Investment Grade
5       A      6      A  Investment Grade
6      A-      7      A  Investment Grade
7    BBB+      8    BBB  Investment Grade
8     BBB      9    BBB  Investment Grade
9    BBB-     10    BBB  Investment Grade
10    BB+     11     BB        High Yield
```

Column	Holds
`score`	week 1's number: 1 for AAA, one more per notch down
`bucket`	the notches grouped: AA+, AA, and AA- are all AA
`grade`	Investment Grade down to BBB-, score 10; High Yield from BB+, score 11

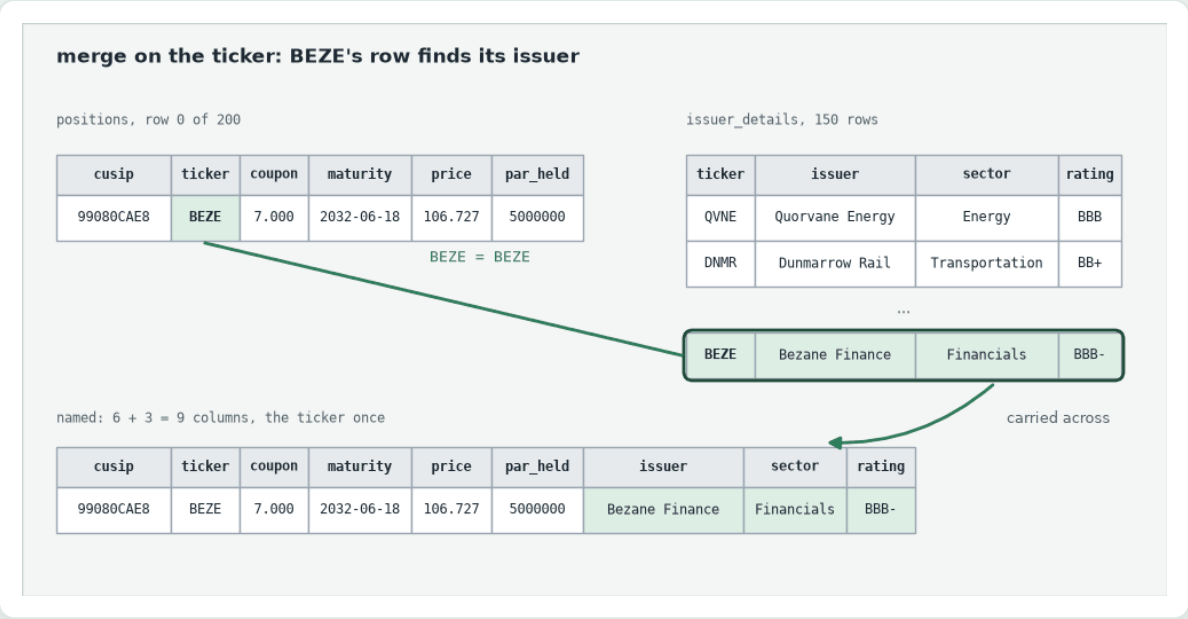
Many rows on one side, one on the other: the shape of a lookup.

8.2 One merge puts three columns on 200 positions

```
issuer_details = issuers[['ticker', 'issuer', 'sector', 'rating']]
named = pd.merge(positions, issuer_details, on='ticker')
print(positions.shape, issuer_details.shape, named.shape)
named.head(3)
(200, 6) (150, 4) (200, 9)
       cusip ticker  coupon    maturity    price  par_held  \
0  99080CAE8   BEZE   7.000  2032-06-18  106.727   5000000
1  99049XAE2   BEZH   8.125  2029-04-17   97.556    250000
2  99039NAC0   BEZL   5.125  2030-06-03   98.730   1500000

              issuer          sector rating
0     Bezane Finance      Financials   BBB-
1  Bezamyth Machinery     Industrials      B
2    Bezoquil Freight  Transportation     BB
```

In Excel this is a "VLOOKUP" on the ticker, filled down 200 rows, once for each column wanted.



"on=" names the column to match on: the key.

200 in, 200 out, and three columns more.

A second lookup: score, grade, credit order

```
rated = pd.merge(named, ratings[['rating', 'score', 'grade']], on='rating')
print(rated.shape)
rated[['ticker', 'issuer', 'rating', 'score', 'grade']].head(3)
(200, 11)
  ticker             issuer rating  score            grade
0   BEZE      Bezane Finance   BBB-     10  Investment Grade
1   BEZH  Bezamyth Machinery      B     15        High Yield
2   BEZL    Bezoquil Freight     BB     12        High Yield
```

```
rated['grade'].value_counts()
grade
Investment Grade    128
High Yield           72
Name: count, dtype: int64
```

The last notebook's text sort put AAA seventh. A number sorts the way a rating should.

```
print(rated.sort_values('score')['rating'].unique().tolist())
['AAA', 'AA+', 'AA', 'AA-', 'A+', 'A', 'A-', 'BBB+', 'BBB', 'BBB-', 'BB+', 'BB', 'BB-', 'B+', 'B', 'B-', 'CCC+', 'CCC']
```

Excel side by side. Ticker in column B of the positions sheet; on the issuers sheet, ticker in A, sector in C

Job	pandas	Excel
Bring in the sector	`pd.merge(positions, issuers[['ticker', 'sector']], on='ticker')`	=VLOOKUP(B2, issuers!A:C, 3, FALSE) filled down
Without counting columns	`the same line`	=XLOOKUP(B2, issuers!A:A, issuers!C:C) in Excel 2021 or later and Microsoft 365;
		INDEX and MATCH in any version

Three tables answered what one file could not: 128 investment grade, 72 high yield.

8.3 Keep every position, then see what did not match

```
desk_small = pd.DataFrame({'ticker': ['QVNE', 'DNMR', 'PLWK', 'HLVF'],
                           'face': [1500000, 500000, 750000, 250000]})
sector_small = pd.DataFrame({'ticker': ['QVNE', 'DNMR', 'PLWK', 'THAR'],
                             'sector': ['Energy', 'Transportation', 'Materials',
                                        'Materials']})
```

```
  ticker     face
0   QVNE  1500000
1   DNMR   500000
2   PLWK   750000
3   HLVF   250000
  ticker          sector
0   QVNE          Energy
1   DNMR  Transportation
2   PLWK       Materials
3   THAR       Materials
```

A left merge is the closest to "VLOOKUP": every row of your sheet stays. Outer sorts the rows by ticker; "_merge" reads "both", "left_only", or "right_only", the fastest view of what did not match.

```
left_join = pd.merge(desk_small, sector_small, on='ticker', how='left')
left_join
```

```
  ticker     face          sector
0   QVNE  1500000          Energy
1   DNMR   500000  Transportation
2   PLWK   750000       Materials
3   HLVF   250000             NaN
```

```
no_sector = left_join['sector'].isna()
print('No sector:', no_sector.sum())
left_join[no_sector]
```

```
No sector: 1
  ticker    face sector
3   HLVF  250000    NaN
```

```
pd.merge(desk_small, sector_small, on='ticker', how='outer', indicator=True)
```

```
  ticker       face          sector      _merge
0   DNMR   500000.0  Transportation        both
1   HLVF   250000.0             NaN   left_only
2   PLWK   750000.0       Materials        both
3   QVNE  1500000.0          Energy        both
4   THAR        NaN       Materials  right_only
```

For details on a list of positions, use "left".

Refuse a repeated key, then guard the real files

```
pd.merge(desk_small, sector_twice, on='ticker', how='left', validate='many_to_one')
MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
```

```
guarded = pd.merge(positions, issuer_details, on='ticker', how='left',
                   validate='many_to_one')
assert guarded.shape[0] == positions.shape[0], 'the merge changed the row count'
```

```
matched = pd.merge(positions, issuer_details, on='ticker', how='left', indicator=True)
matched['_merge'].value_counts()
_merge
both          200
left_only       0
right_only      0
Name: count, dtype: int64
```

Every lookup today is many to one: many positions, one row per key. Silent: the guard passed, and it runs again on next month's file.

"validate=" refuses a repeated key; "isna()" and the "_merge" counts find a missing one.

Excel side by side

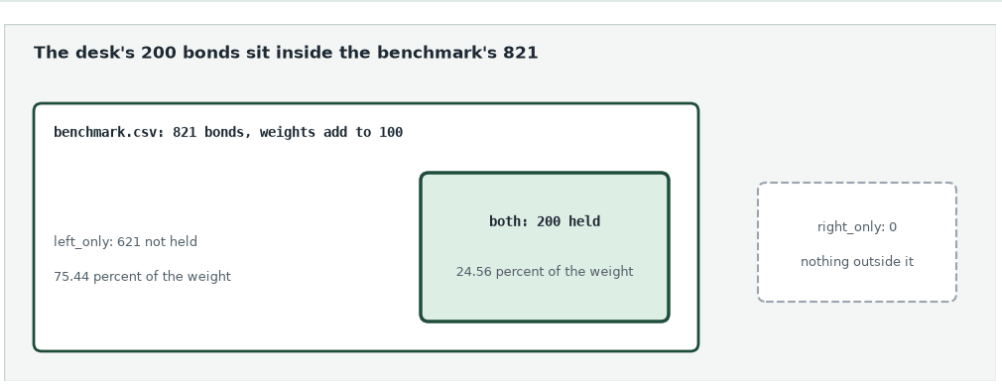
Job	pandas	Excel
A key with no match	`NaN, found with isna()`	#N/A, found with =ISNA(...)
Choose the join	how=	Power Query's Merge Queries (Excel 2016 on):
		Join Kind Inner, Left Outer, Right Outer, or Full Outer

8.4 Benchmark bonds held, and the weight they carry

```
desk_face = holdings[['cusip', 'par_held']]
compared = pd.merge(benchmark, desk_face, on='cusip', how='left')
print(compared.shape)
compared[['cusip', 'ticker', 'weight_pct', 'par_held']].head(4)
(821, 20)
       cusip ticker  weight_pct   par_held
0  99001BAA4   QVNE      0.2076  2750000.0
1  99001BAB2   QVNE      0.1551        NaN
2  99001BAC0   QVNE      0.0245        NaN
3  99001BAD8   QVNE      0.2024        NaN
```

```
held = compared['par_held'].notna()
not_held = compared['par_held'].isna()
print('Held:', held.sum(), 'Not held:', not_held.sum())
print('Weight held:', round(compared.loc[held, 'weight_pct'].sum(), 2))
print('Weight not held:', round(compared.loc[not_held, 'weight_pct'].sum(), 2))
Held: 200 Not held: 621
Weight held: 24.56
Weight not held: 75.44
```

```
both_files = pd.merge(benchmark, desk_face, on='cusip', how='outer', indicator=True)
both_files['_merge'].value_counts()
_merge
left_only     621
both          200
right_only      0
Name: count, dtype: int64
```



Excel side by side

On the benchmark sheet, a helper column T with =COUNTIF(holdings!B:B, B2) gives 1 for a bond held and 0 for one not; =SUMIFS(S:S, T:T, 1) adds the weights held, column S.

200 of 821 bonds, 24.56 percent of the weight, nothing outside it.

8.5 Two traders' tables stacked, and side by side

```
book = pd.merge(holdings, ratings, on='rating')
ig_book = book[book['grade'] == 'Investment Grade']
hy_book = book[book['grade'] == 'High Yield']
stacked = pd.concat([ig_book, hy_book])
print(ig_book.shape, hy_book.shape, stacked.shape)
print('Market value:', stacked['market_value'].sum())
(128, 22) (72, 22) (200, 22)
Market value: 490819215.0
```

```
print(stacked.index[126:130].tolist())
stacked = pd.concat([ig_book, hy_book], ignore_index=True)
print(stacked.index[126:130].tolist())
[198, 199, 1, 2]
[126, 127, 128, 129]
```

```
price_part = book[['ticker', 'price']].head(3)
oas_shifted = book[['oas']].iloc[1:4]
print(pd.concat([price_part, oas_shifted], axis=1))
  ticker    price    oas
0   BEZE  106.727    NaN
1   BEZH   97.556  535.0
2   BEZL   98.730  153.0
3    NaN      NaN  218.0
```



Excel side by side

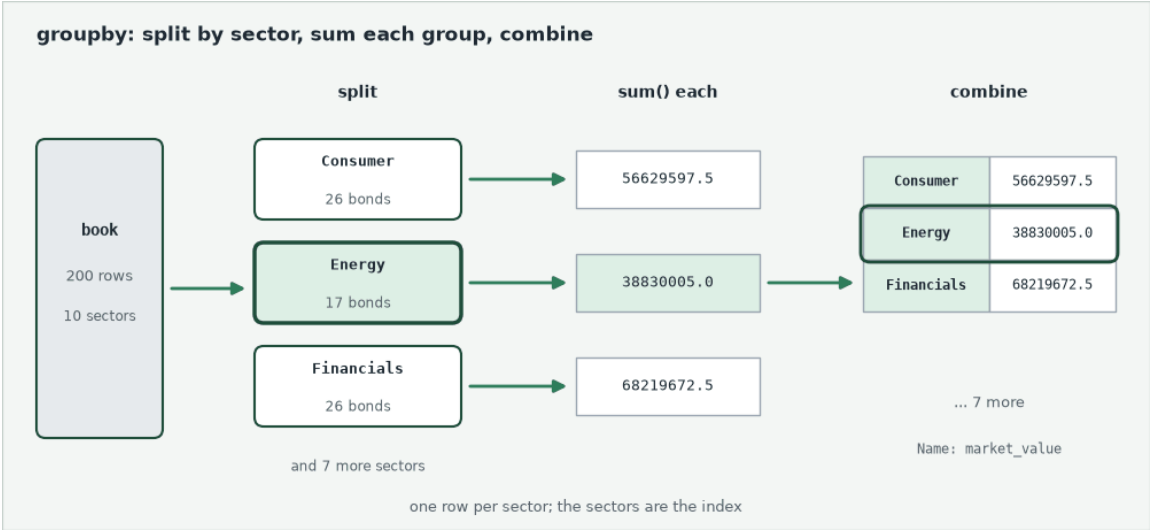
Pasting one range under another, or =VSTACK(range1, range2) in Microsoft 365 and Excel 2024, is concat; =HSTACK(range1, range2) sets ranges side by side, as axis=1 does. Excel lines ranges up by position, so HSTACK would set row 1's spread beside row 0's price; pandas lines them up by label.

A key column: "merge". Row labels, or one table under another: "concat".

8.6 groupby: market value in every sector at once

```
sector_value = book.groupby('sector')['market_value'].sum()
sector_value
sector
Consumer          56629597.5
Energy            38830005.0
Financials        68219672.5
Health Care       38908187.5
Industrials       52540745.0
Materials         71701617.5
Technology        30311760.0
Telecom           55848270.0
Transportation    48822897.5
Utilities         29006462.5
Name: market_value, dtype: float64
```

```
top_three = sector_value.sort_values(ascending=False).head(3)
print(top_three)
print(top_three.index.tolist())
sector
Materials     71701617.5
Financials    68219672.5
Consumer      56629597.5
Name: market_value, dtype: float64
['Materials', 'Financials', 'Consumer']
```




```
book.groupby('grade')['oas'].agg(['count', 'mean', 'median', 'min', 'max']).round(2)
                count    mean  median  min   max
grade
High Yield         72  339.38   250.5  125  1551
Investment Grade  128   99.34    95.0   19   204
```

One line does the last notebook's filter for every sector.

The sector summary, and the PivotTable beside it

```
sector_summary = book.groupby('sector').agg(
    bonds=('cusip', 'count'),
    market_value=('market_value', 'sum'),
    avg_oas=('oas', 'mean'),
    avg_duration=('duration', 'mean'),
)
```

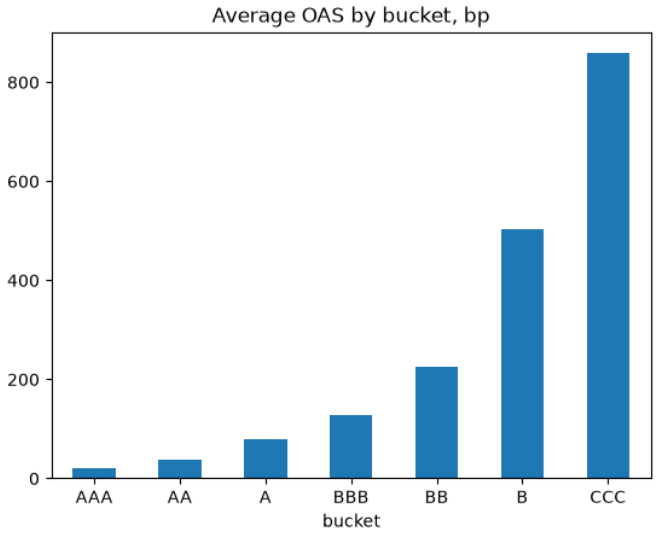
The pivot table's twin, styled as in 7.10. Technology is widest, 298.17 bp on 12 bonds; Transportation is tightest, 126.5, and longest, 8.19 years.

sector	bonds	market_value	avg_oas	avg_duration
Consumer	26	56,629,598	218.88	6.84
Energy	17	38,830,005	171.59	6.97
Financials	26	68,219,672	172.31	6.15
Health Care	20	38,908,188	141.75	5.87
Industrials	20	52,540,745	220.55	6.33
Materials	26	71,701,618	182.42	5.61
Technology	12	30,311,760	298.17	5.75
Telecom	21	55,848,270	182.38	6.35
Transportation	20	48,822,898	126.50	8.19
Utilities	12	29,006,462	177.92	6.82

Excel side by side. Sector in column E, OAS in M, market value in S, as in holdings.csv

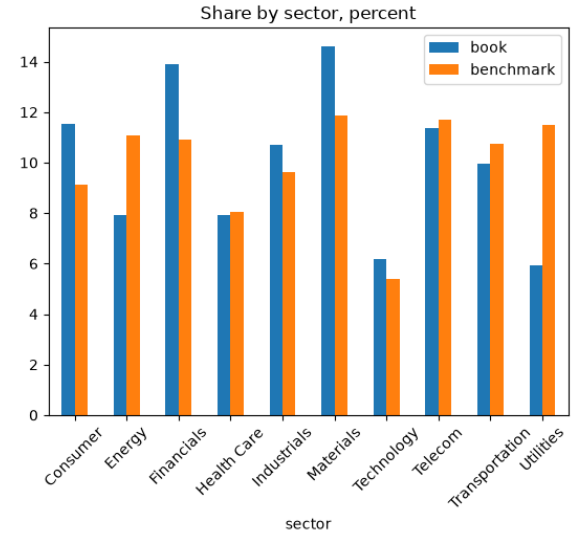
Question	pandas	Excel
Market value of one sector	`sector_value.loc['Energy']`	=SUMIFS(S:S, E:E, "Energy")
Average OAS of one sector	`book.groupby('sector')['oas'].mean().loc['Energy']`	=AVERAGEIFS(M:M, E:E, "Energy")
Bonds in one sector	`book.groupby('sector')['cusip'].count().loc['Energy']`	=COUNTIFS(E:E, "Energy")
Every sector at once	`book.groupby('sector')['market_value'].sum()`	a PivotTable: sector in Rows, Sum of market_value in Values
Several statistics at once	`agg(...)`	more fields in Values

Spread by bucket, and the book against the benchmark




```
bucket_oas = book.groupby('bucket')['oas'].mean()
credit_order = ['AAA', 'AA', 'A', 'BBB', 'BB', 'B', 'CCC']
chart = bucket_oas.loc[credit_order].plot.bar(rot=0, title='Average OAS by bucket, bp')
```

Each bucket is wider than the one above it. A's 79.41 is 7.7's single-A "isin()" figure, now one group of seven.




```
book_pct = sector_value / sector_value.sum() * 100
bench_pct = benchmark.groupby('sector')['weight_pct'].sum()
shares = pd.DataFrame({'book': book_pct, 'benchmark': bench_pct})
chart = shares.plot.bar(rot=45, title='Share by sector, percent')
```

The two largest gaps: Utilities, 5.91 percent of the book against 11.50 of the benchmark, and Energy, 7.91 against 11.06.

Predict first: decide, then run each in the notebook

Predict 1

The rating scale is keyed on the rating. What does "pd.merge(positions, ratings, on='rating')" give? A) 200 rows, with the score, bucket, and grade added. B) Fewer than 200 rows: some ratings find no match. C) An error.

```
pd.merge(positions, ratings, on='rating')
```

Predict 3

How many rows does "pd.merge(desk_small, sector_small, on='ticker')" return? A) Five: every ticker from both tables. B) Four: every bond the desk holds. C) Three: the tickers in both tables.

```
pd.merge(desk_small, sector_small, on='ticker')
```

Predict 2

Another issuer file calls its key "issuer_ticker". "on='ticker'" gave nine columns, the key once. How many does "left_on='ticker', right_on='issuer_ticker'" give? A) Nine. B) Ten.

```
other_names = issuer_details.rename(columns={'ticker': 'issuer_ticker'})
different_keys = pd.merge(positions, other_names, left_on='ticker', right_on='issuer_ticker')
print(different_keys.shape)
```

Predict first: a repeated key, a shared column

Predict 4

A sector table lists QVNE twice. The desk's face adds to 3,000,000. What does it add to after a left merge? A) 3,000,000: a left merge keeps the desk's four rows. B) 4,500,000: QVNE's 1,500,000 counted twice. C) An error: a key is listed twice.

```
sector_twice = pd.DataFrame({'ticker': ['QVNE', 'QVNE', 'DNMR', 'PLWK', 'HLVF'],
                             'sector': ['Energy', 'Energy', 'Transportation', 'Materials',
                                        'Consumer']})
doubled = pd.merge(desk_small, sector_twice, on='ticker', how='left')
print(doubled)
print('Face before:', desk_small['face'].sum(), 'after:', doubled['face'].sum())
```

Predict 5

Both files have a "price" column besides the key. What does "pd.merge(benchmark[['cusip', 'price']], holdings[['cusip', 'price']], on='cusip')" give? A) An error: two columns would share one name. B) One "price" column, from the left table. C) Two price columns, under new names.

```
same_name = pd.merge(benchmark[['cusip', 'price']], holdings[['cusip', 'price']], on='cusip')
same_name.head(2)
```

The note, answered

The note asked	Answer
A name, sector, and rating on every position	three files, two keys: 200 in, 200 out
None lost or doubled	validate=, an assert, and all 200 both
Benchmark bonds held, and their weight	200 of 821, 24.56 percent of the weight
The book by sector, in money, beside the benchmark	groupby, and the share chart

Be proud: a "VLOOKUP", a stack, and a pivot table are now lines of code that run again on next month's files.

Every average today counted each bond once. Part 2 weights them by market value, saves the book to a file and a workbook, and turns its maturity dates into a profile.